Microcontrollers

Rex Joseph
Department of Electrical and
Electronics Engineering

What do these have in common?

Embedded Systems

Micro-controllers are the heart of **embedded systems**

Systems that work in real-time

Usually perform control tasks

Embedded systems are present in almost any device

More than 100 in a modern day automobile!

VCU

Power steering

ABS

HVAC

…..

Embedded Systems

Micro-controllers are the heart of **embedded systems**

Systems that work in real-time

Usually perform control tasks

Embedded systems are present in almost any device

More than 100 in a modern day automobile!

VCU

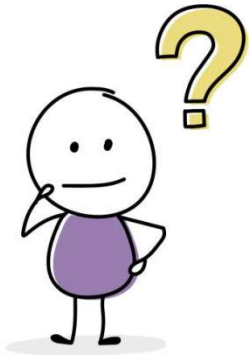
Power steering

ABS

HVAC

…..

What is a micro-controller ?



Recap: Finite State Machines

In information technology and computer science, a system is described as **stateful** if it is designed to remember preceding events or user interactions

The remembered information is called the **state** of the system.

The set of states a system can occupy is known as its state space.

In a discrete system, the state space is countable and often finite.

Source: Wikipedia

Recap: Finite State Machines

The system's internal behaviour or **interaction with its environment** consists of **separately occurring individual actions or events**, such as accepting input or producing output, that **may or may not** cause the system to change its state.

Examples of such systems are digital logic circuits and components, automata and formal language, computer programs, and computers.

The output of a digital circuit or deterministic computer program at any time is completely determined by its current inputs and its state.

The state of a microprocessor is the contents of all the memory elements in it: the accumulators, storage registers, data caches, and flags.

Source: Wikipedia

A very simple microprocessor – only adds!

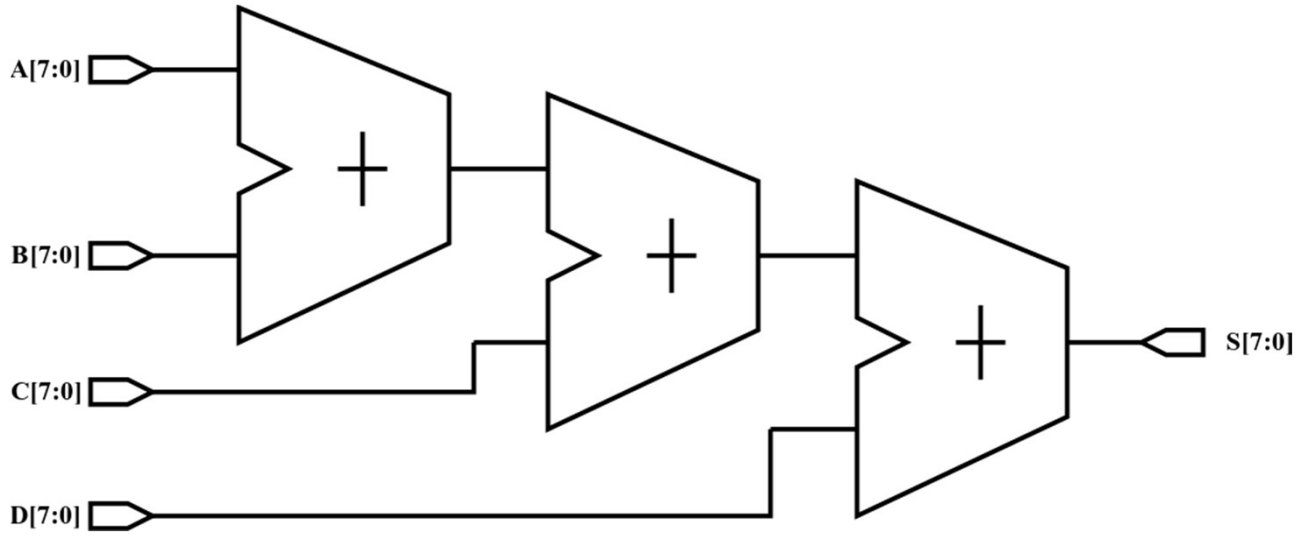
But gives us some insight into the uP

Example: S = a + b + c + d (4 inputs, one output)

How do we do it? (Using two input adders, of course!)

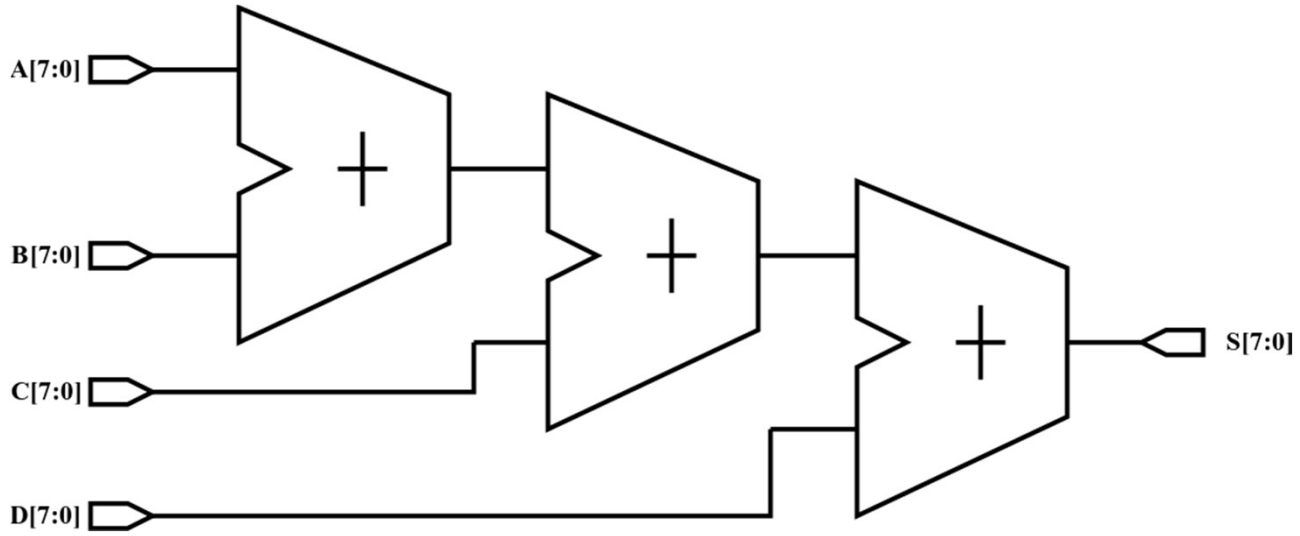
(Just conceptual, carry in, out, not considered)

A very simple microprocessor



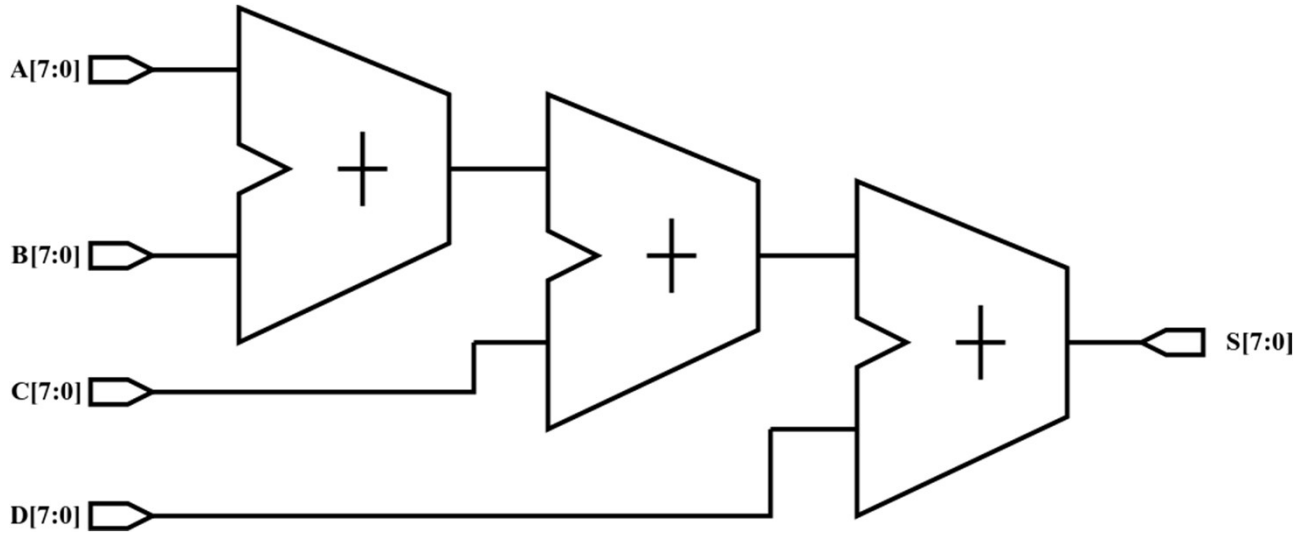
Is this sequential or combinational?
Drawbacks?

A very simple microprocessor



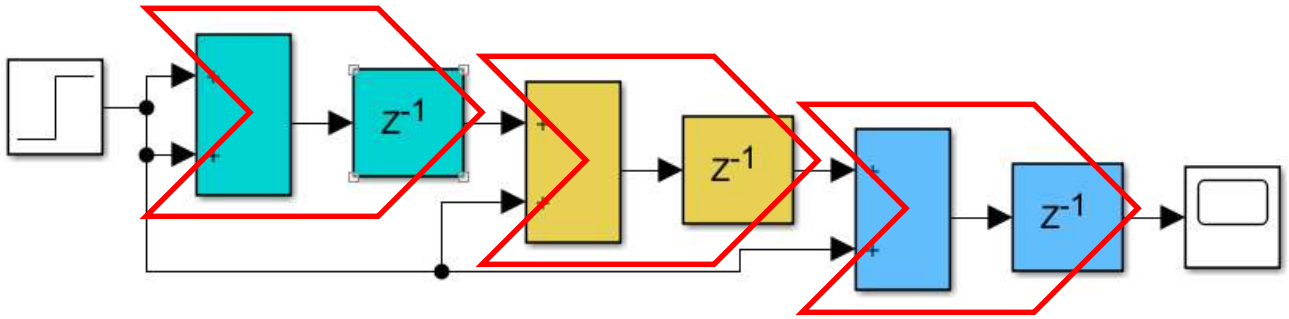
What happens at the output when input/s change?
What if we wanted to add 10 inputs?
Alternative approach?

A very simple microprocessor



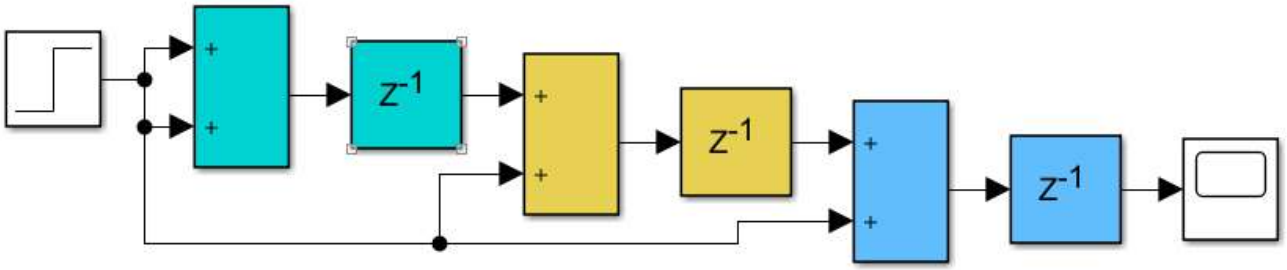
What happens at the output when input/s change?
What if we wanted to add 10 inputs? Add more stages! Not practical
Alternative approach?

A very simple microprocessor



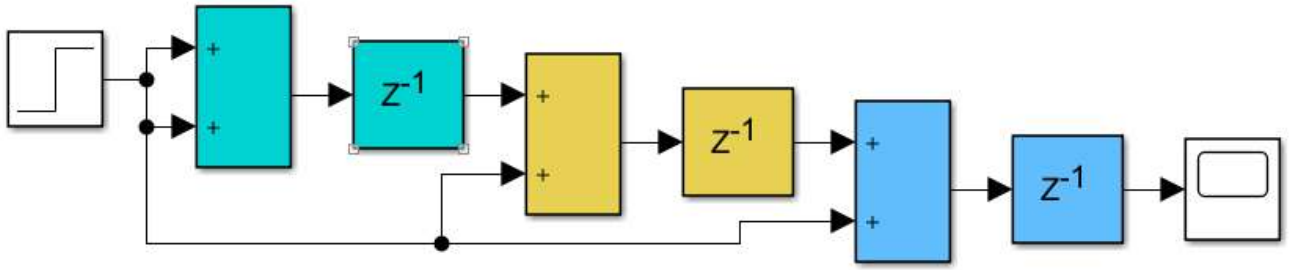
What happens at the output when input/s change?
What if we wanted to add 10 inputs? Add more stages! Not practical
Alternative approach?

A very simple microprocessor

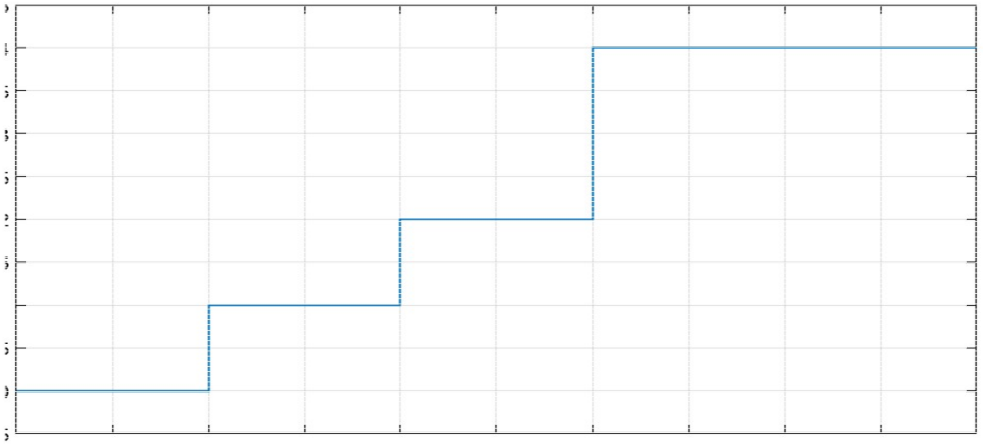


Combinational delay of 1 second (we need time), input changes from 0 to 1 at t=0
Plot the output

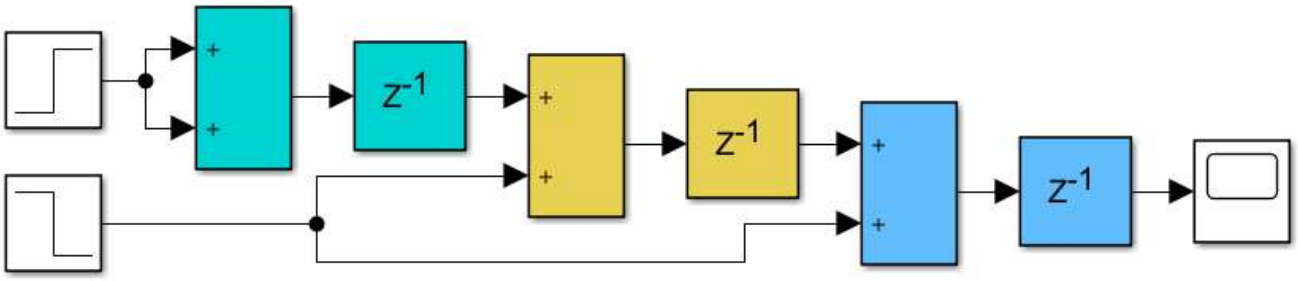
A very simple microprocessor



Combinational delay of 1 second (we need time), input changes from 0 to 1 at t=0
Plot the output



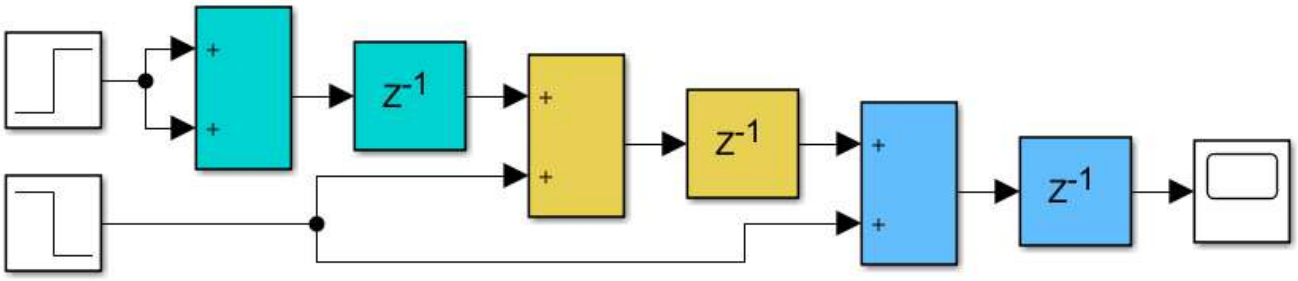
A very simple microprocessor



What about this? A, B 0 -> 1, C, D 1 -> 0, at t=1

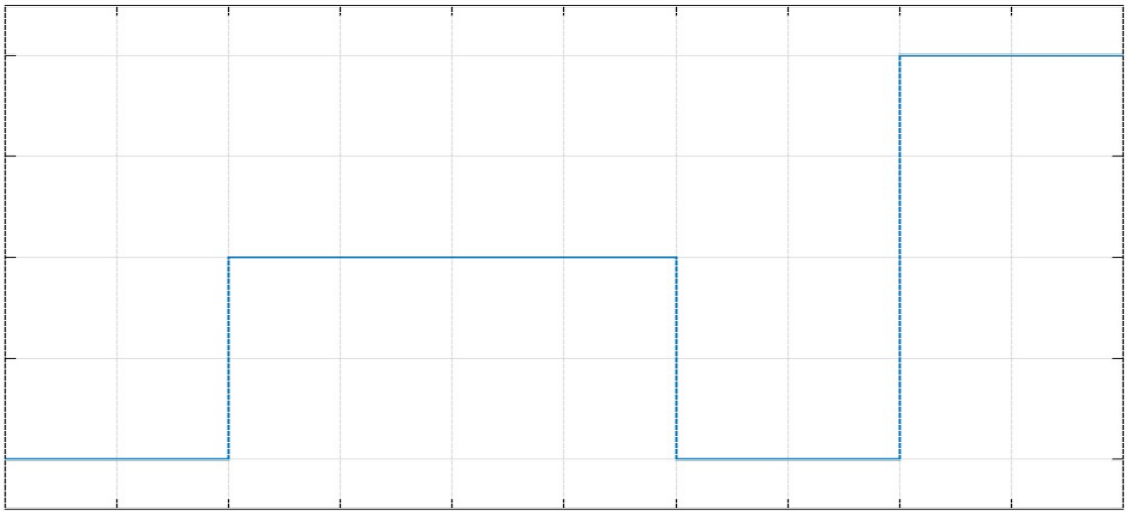
Plot the output

A very simple microprocessor



What about this? A, B 0 -> 1, C, D 1 -> 0, at t=1

Plot the output



A very simple microprocessor – only adds!

Output settles down finally

How do we know how much time?

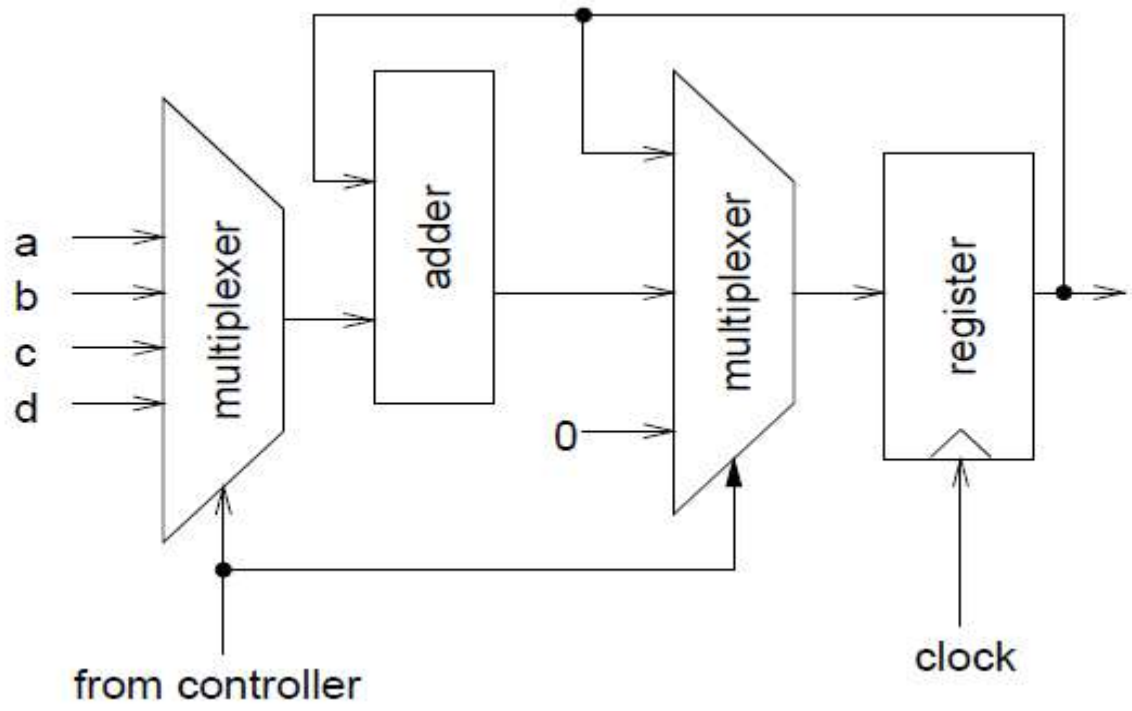
What happens if some forbidden (incorrect) value in between

We can’t keep adding adders!!

How about sequential implementation?

A very simple microprocessor – only adds!

Sequential implementation?



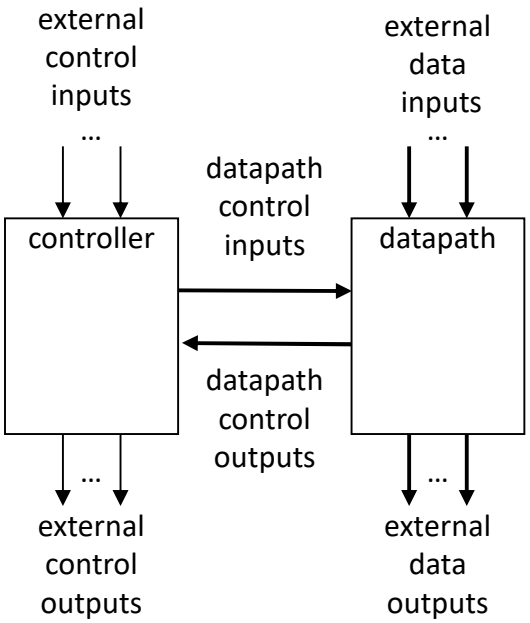
A very simple microprocessor – only adds!

Reset to zero. Hold zero by selecting output registers to loop
Add first input by selecting first input and adder options in muxes
Hold result till next cycle and so on.

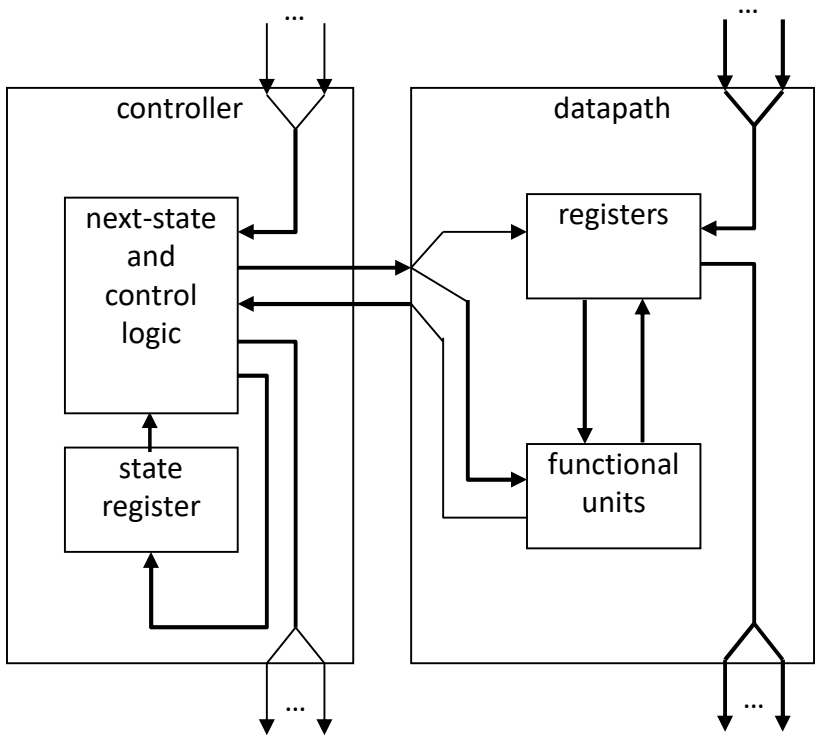
We have the correct result after a certain number of clock cycles
In between values need not be considered
Only one adder
The multiplexer at input can be replaced by memory fetch
This will get any number of variables to be added

Key components:
Controller that determines what actions need to be done
Registers that hold data values and combinational circuits to perform operations.
Controller plus data-path – FSMD! - and we can build processors!!

An abstraction of a microprocessor!

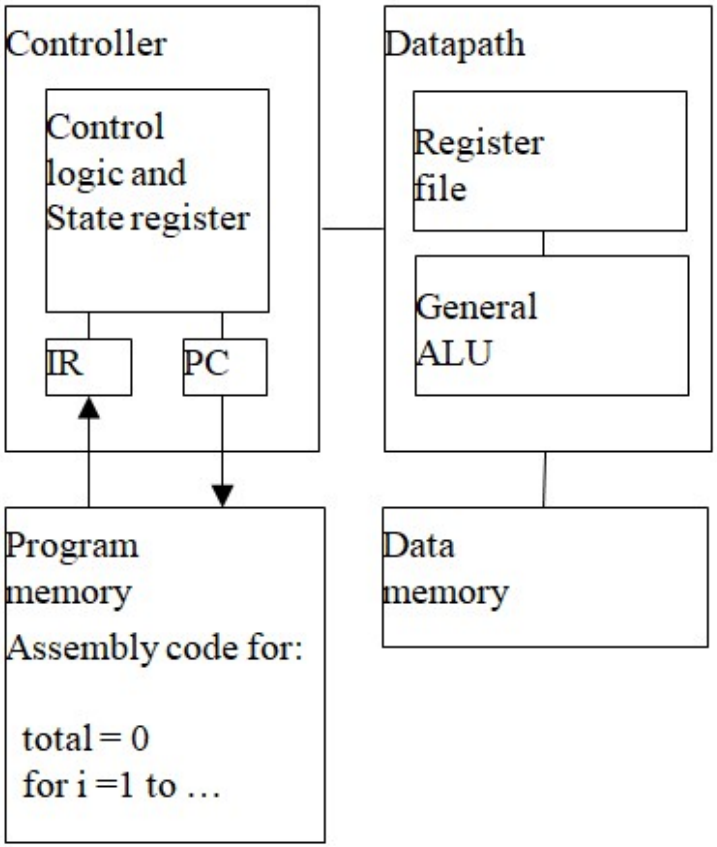


controller and datapath



a view inside the controller and datapath

A microprocessor



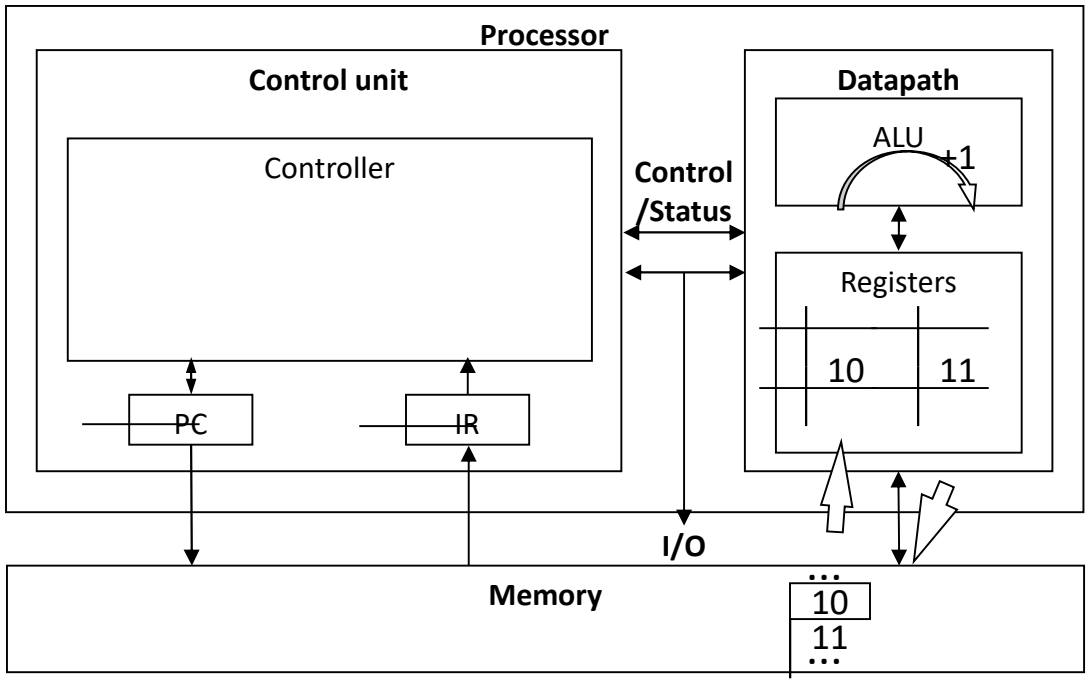
Two important units in controller Program counter and Instruction register
The Arithmetic and Logic Unit in the data-path.

What operation controller needs to perform on the data is coded in memory – program memory

Data is stored in memory too.

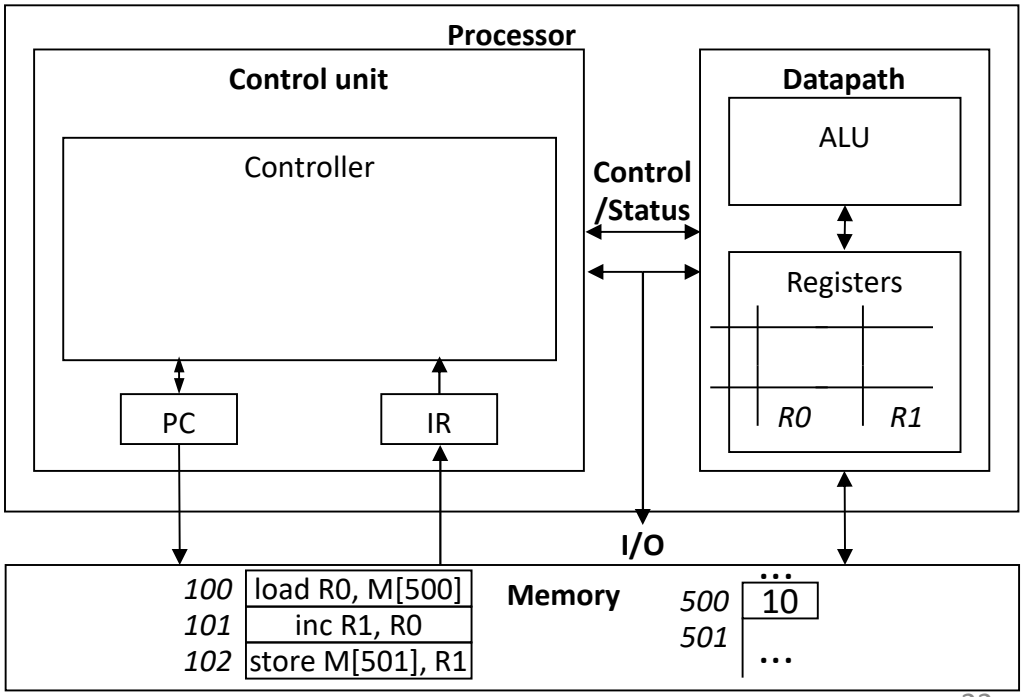
Data-path Operations

- Load
  - Read memory location into register
- ALU operation
  - Input certain registers through ALU, store back in register
- Store
  - Write register to memory location



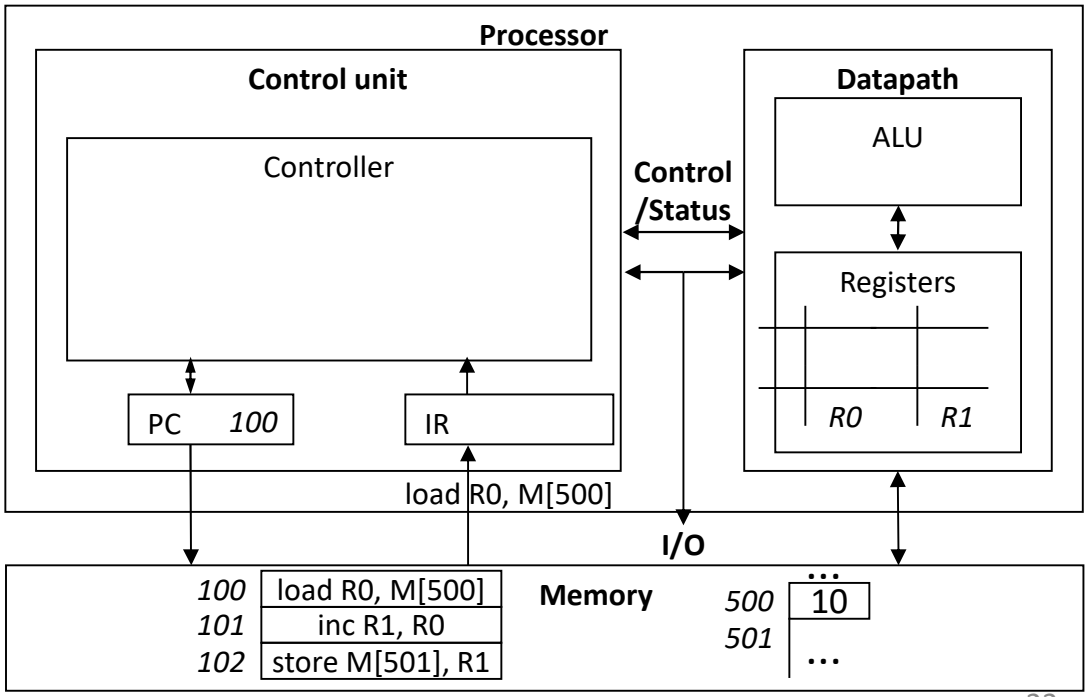
Control Unit

- Control unit: configures the datapath operations
  - Sequence of desired operations ("instructions") stored in memory – "program"
- Instruction cycle – broken into several sub-operations, each one clock cycle, e.g.:
  - Fetch: Get next instruction into IR
  - Decode: Determine what the instruction means
  - Fetch operands: Move data from memory to data-path register
  - Execute: Move data through the ALU
  - Store results: Write data from register to memory



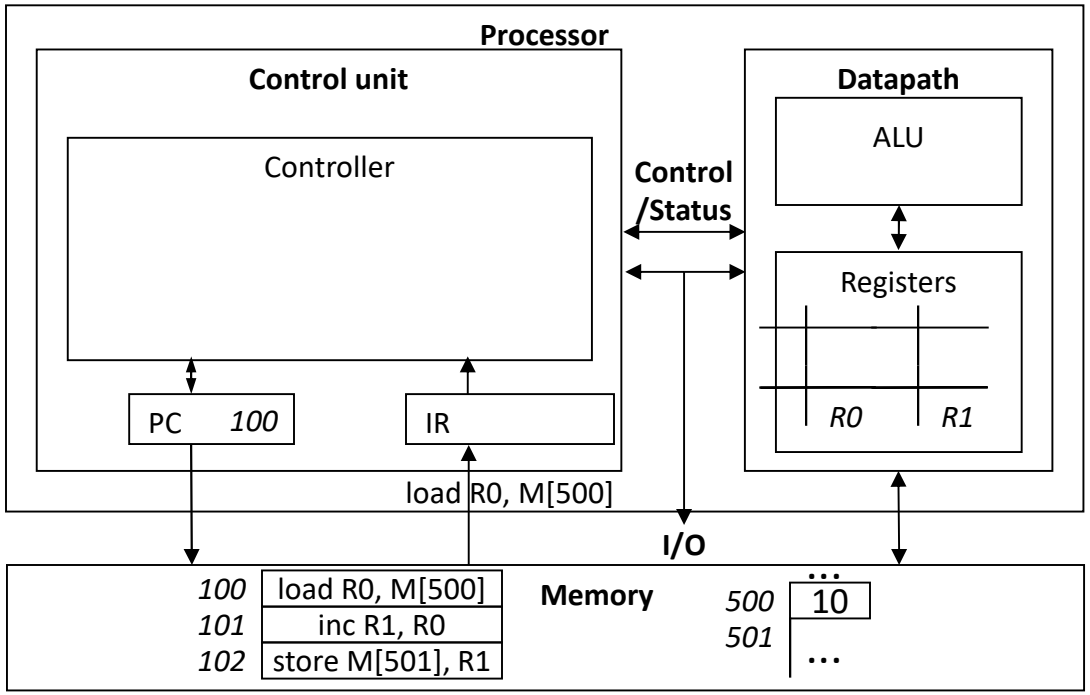
Control Unit Sub-Operations

- Fetch
  - Get next instruction into IR
  - PC: program counter, always points to next instruction
  - IR: holds the fetched instruction



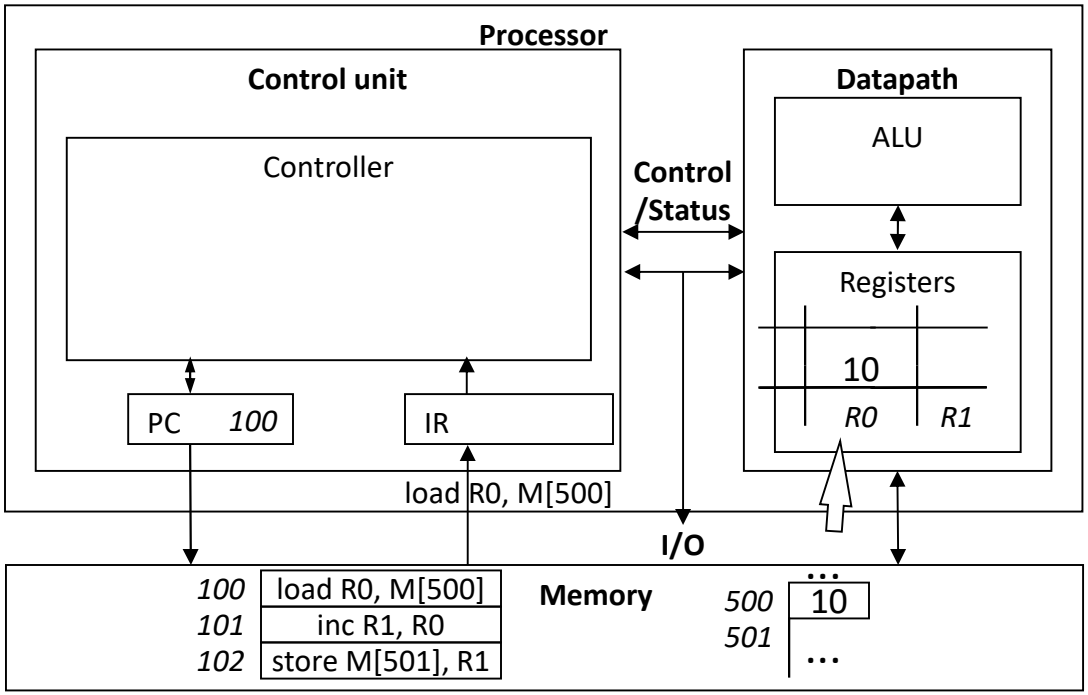
Control Unit Sub-Operations

- Decode
  - Determine what the instruction means



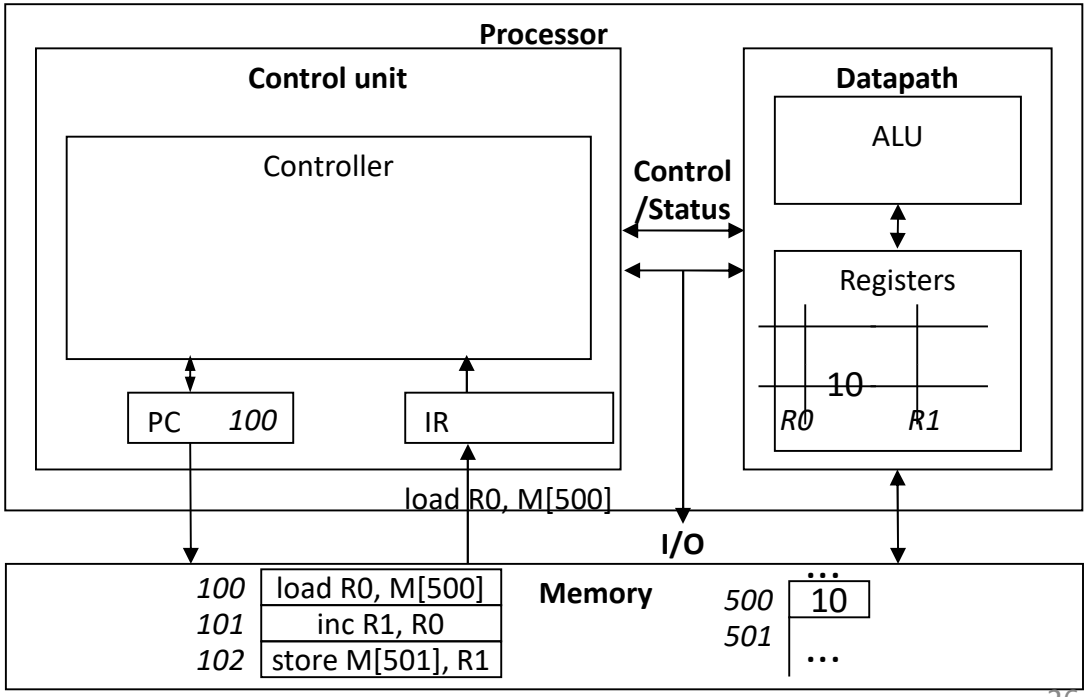
Control Unit Sub-Operations

- Fetch operands
  - Move data from memory to datapath register



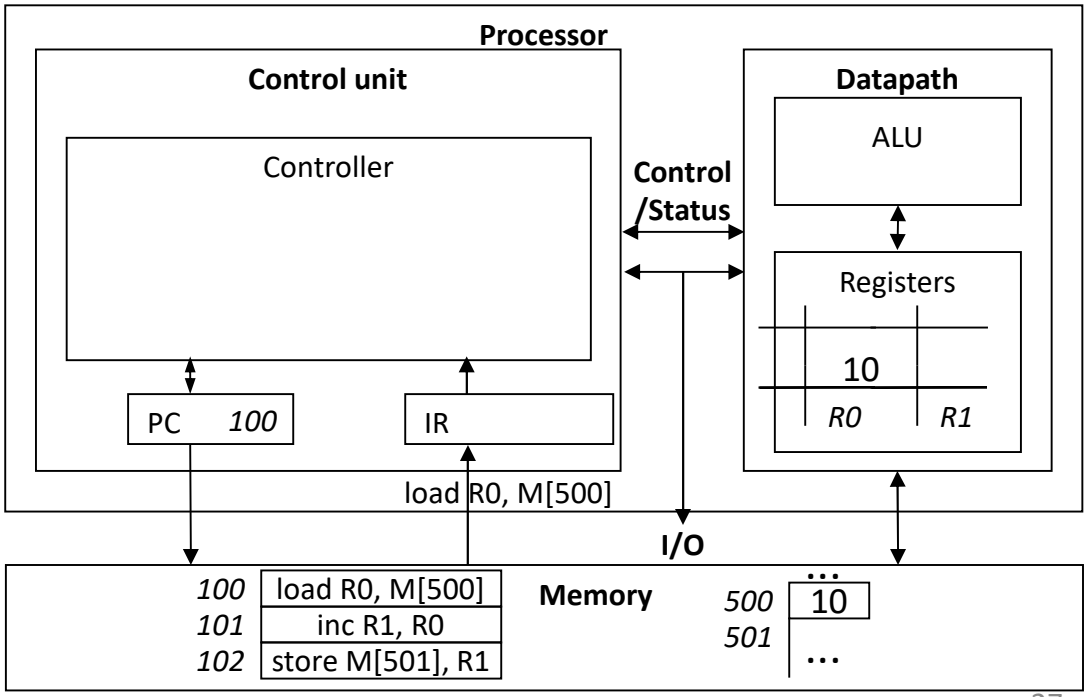
Control Unit Sub-Operations

- Execute
  - Move data through the ALU



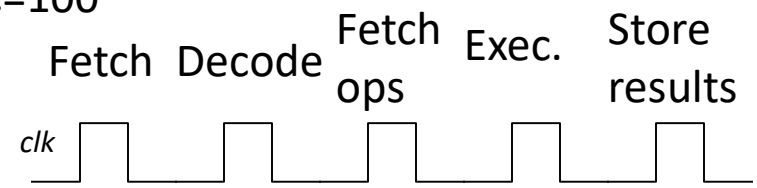
Control Unit Sub-Operations

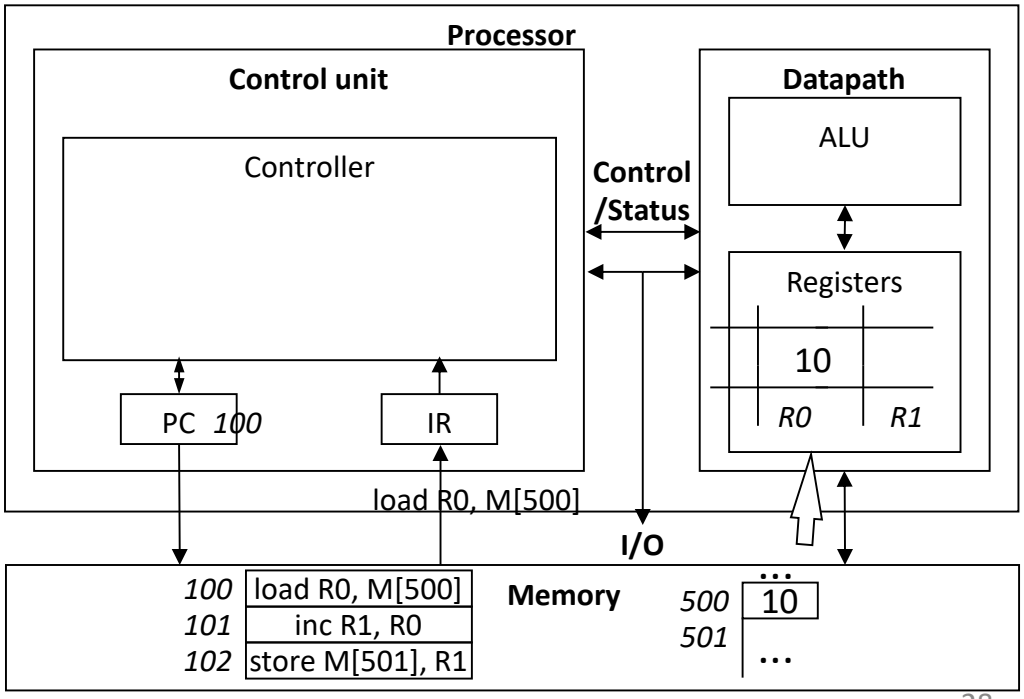
- Store results
  - Write data from register to memory



Instruction Cycles

PC=100





Instruction Cycles

PC=100

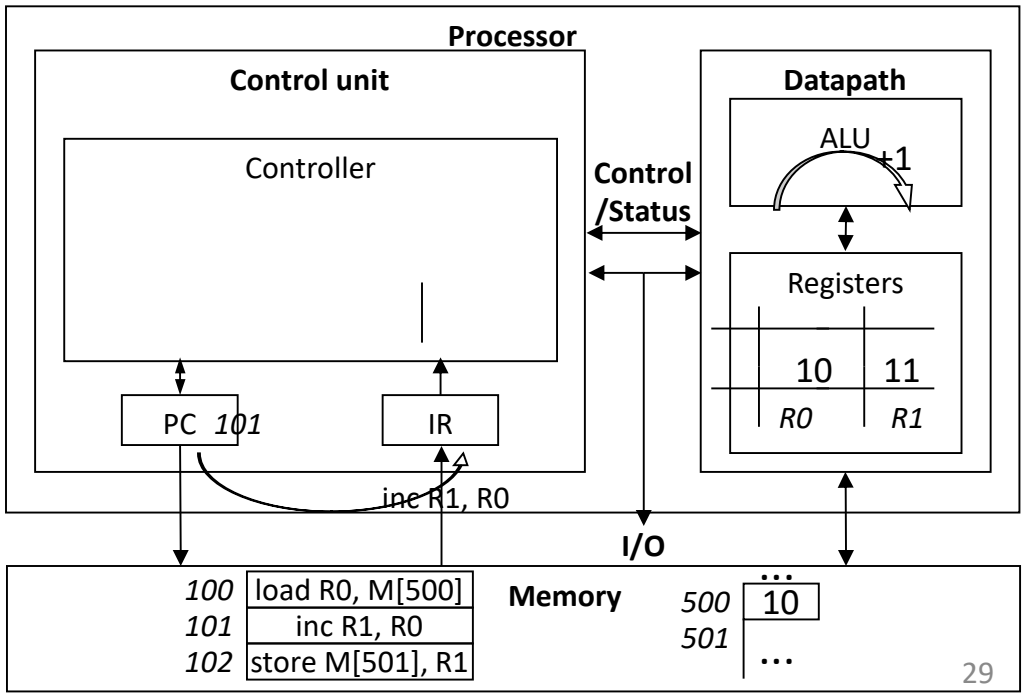
Fetch Decode Fetch ops Exec. Store results

clk

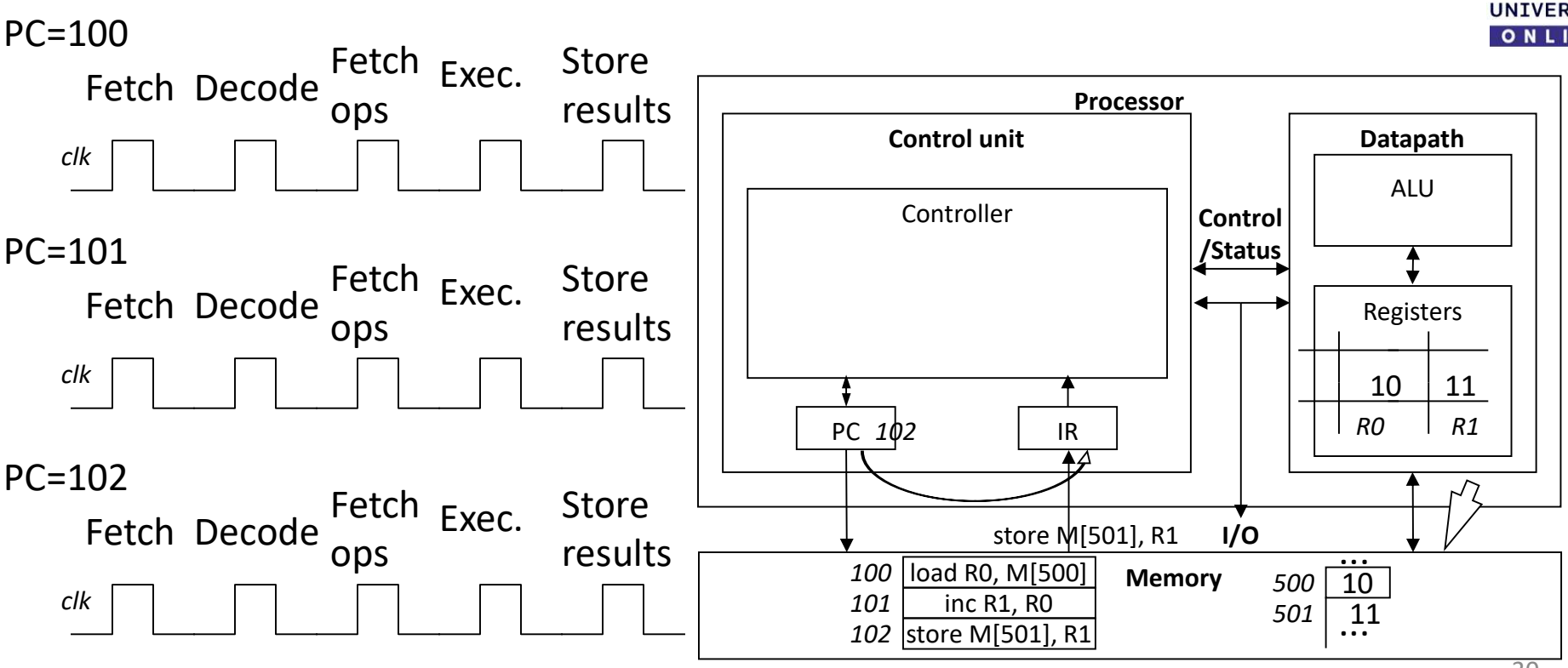
PC=101

Fetch Decode Fetch ops Exec. Store results

clk



Instruction Cycles



Some lose ends!

Microprocessor – but the course is on microcontroller

Microprocessor has limited (cache) or no internal memory.
Needs external memory to store program and data
Used for applications like desktops, laptops etc and doesn't need to control any process.

Microcontroller is a small computer on chip for control applications

Enough memory to run fairly large applications
Input/output ports with analog inputs in all, outputs in some
Integrated **peripherals** like ADC, timers etc

At **reset or startup**, the PC always points to a particular fixed address.
The first instruction it executes after power ON/reset is located here.
Called start-up or BOOT code. (Could be a jump instruction to boot code)

Some lose ends!

Microprocessor – but the course is on microcontroller

Microprocessor has limited (cache) or no internal memory.
Needs external memory to store program and data
Used for applications like desktops, laptops etc and doesn't need to control any process.

Microcontroller is a small computer on chip for control applications

Enough memory to run fairly large applications
Input/output ports with analog inputs in all, outputs in some
Integrated **peripherals** like ADC, timers etc

At **reset or startup**, the PC always points to a particular fixed address.
The first instruction it executes after power ON/reset is located here.
Called start-up or BOOT code. (Could be a jump instruction to boot code)
A deeper dive as the course progresses – interconnection of basic blocks

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Objectives:

- To Study **TI MSP430** family of micro controllers, their architecture, peripheral features and programming
- To Understand and Provide theoretical and practical aspects of low-power system development using the MSP430
- To Know the peripheral features of the MSP430, which include timers, digital and analog IO and serial communication modules
- To Understand and Present case studies of application of the MSP430 so that the student can handle embedded system design projects independently
- To Know the applications of the MSP430 in embedded systems

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Outcomes:

On completion of this course the students will be able to:

- Implement Assembly programs based on MSP430 microcontroller architecture, its instructions and the addressing modes
- Develop and debug program in C language for specific applications
- Organize the application projects with Software realising using commercial IDE towards deployment
- Organize and develop microcontroller interfaces to various sensors and actuators

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Content:

Unit 1: MSP430 Architecture and ALP

General Concepts of Microcontroller and Embedded Systems. The inside view-Functional block diagram, Memory, Central Processing Unit,
Intro to MSP430 – Architecture, Addressing Modes, Instruction set, Constant Generator and Emulated Instructions, Instruction Cycle and Length, Clock System and Low power modes

Laboratory: Different Program Examples for memory copy, addition and other arithmetic operations, calculations of delay times

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Content:

Unit 2: Embedded C, Digital I/O, Interrupts, Watchdog timer

Embedded C Programming and impact on memory, Interrupts, what happens when an interrupted is requested, Interrupt Service Routines, Digital I/O and interfacing to LEDs and switches, Parallel Ports, Lighting LEDs, Flashing LEDs, Read Input from a Switch, Toggle the LED state by pressing the push button, Parallel Ports, Lighting LEDs, Flashing LEDs, Read Input from a Switch, Toggle the LED state by pressing the push button, LCD interfacing.

Watchdog Timer

Laboratory: Keypad Interface, LCD Interface, LED control, timing

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Content:

Unit 3: Timers, ADC and PWM:

Basic Timer1, Real Time Clock, Timer-A: Timer Block, Capture/Compare Channels, Interrupts from Timer-A, Generation of PWM using Timers, Analog to Digital conversion, characteristics and interfaces using ADC.

Laboratory: interfacing with stepper motor, interfacing with DC motor, Servomotor, sensors. Analog inputs

UE24EE252B: Microcontrollers (4-0-2-6-5)

Course Content:

Unit 4: Serial Communication: UART, I2C, SPI

Asynchronous Serial Communication, Asynchronous Communication with USCI_A, Communications, Peripherals in MSP430, Serial Peripheral Interface, UART functionality and programming, I2C Protocol and speeds, SPI Protocols and Speed.

Laboratory: Communication case studies

UE24EE252B: Microcontrollers (4-0-2-6-5)

Text Book:

1. "MSP430 Microcontroller Basics", John Davies, Newnes (Elsevier Science), 2008.
2. MSP430F2xx, MSP430G2xx Family User's Guide - Texas Instruments

Reference Books:

"MSP430 Microcontroller in Embedded System Project", C P Ravikumar, Elite Publishing House Pvt. Ltd., December 2011.

MSP430 Teaching CD-ROM, Texas Instruments, 2008.

Lab Manual: www.ti.com/lab-manuals Embedded System Design using MSP430 Launchpad Development Kit

UE24EE252B: Microcontrollers (4-0-2-6-5)

Evaluation:
IAS1-20M, ISA2 20M, Assignments 4 M JOB 6M : ESA 50M
Laboratory 10M internals and 10 M ESA:
Total 120 M reduced to 100
Assignments are all in-class supervised assignments

Level 1 problem on hardware 1M
Level 2 problem on Hardware 2M
Level 3 Problem on hardware 3M

THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515